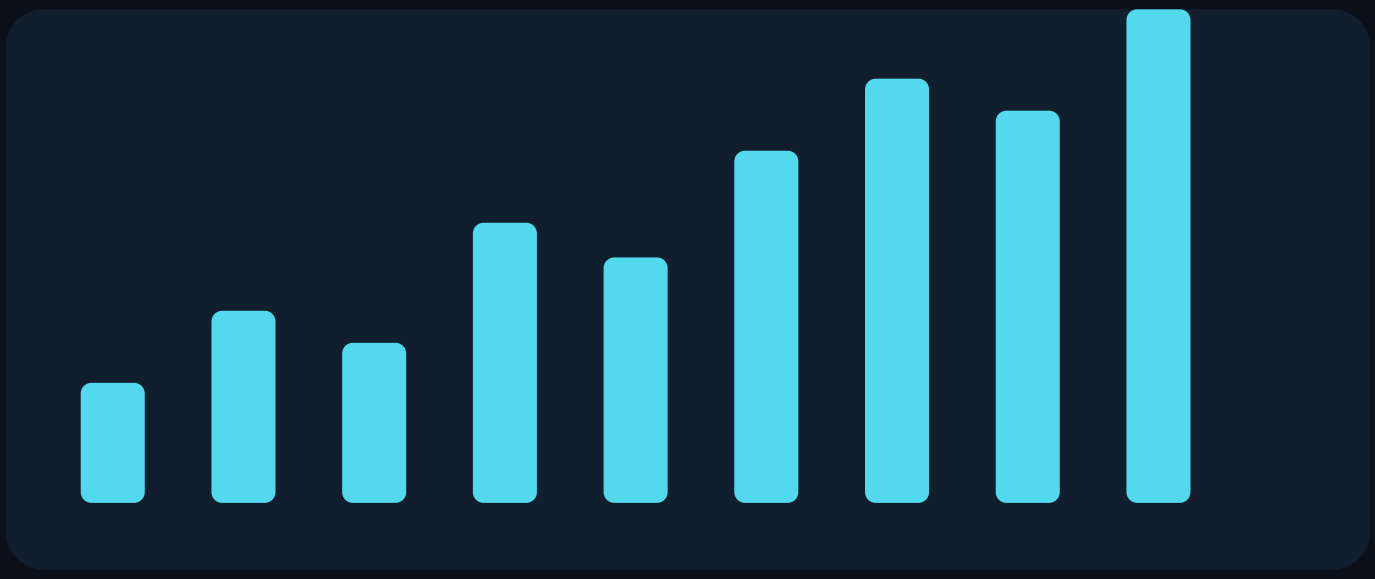
PRACTICAL KNOWLEDGE / DARK EDITION

SQL for Business Analytics

Turn transaction data into reliable business answers. A PostgreSQL practice guide with a complete miniature dataset.

By Datta Sable



DATTA SABLE / PROFESSIONAL FIELD GUIDES

THE READER BRIEF

A guide you can put to work

The difficult part of analytical SQL is rarely remembering SELECT. It is deciding which rows belong in a metric, preserving the grain of the data, and proving that a join has not changed the answer. This guide follows one small retail dataset from raw rows to a reconciled management report.

Read chapters 1-3 first and run the setup script. Then execute each query, predict the result, and compare it with the answer notes. All amounts are invented currency units; shipping and tax are excluded. Revenue means paid order-line amount after line discount. Returns are represented separately as a limitation, not silently assumed away.

You will build a daily revenue report, customer segmentation, ranking and a repeat-purchase calculation. The final deliverable is a compact metric contract with SQL and reconciliation evidence.

What you need

A PostgreSQL-compatible SQL environment and permission to create temporary practice tables. Examples use NUMERIC, DATE, CTEs and window functions. Run them in a practice database, not a production schema.

Edition and scope

Original educational examples by Datta Sable. First edition, 28 October 2026. This purchase contains this PDF only. Code, tables and exercises are printed inside; no separate datasets, software licences or editable files are bundled. Product interfaces and availability can change. Check the linked official documentation when applying a platform-specific step.

Personal learning licence

You may use the techniques in your own work. Do not redistribute, resell or publicly upload the book. The examples use invented data. No customer or employer data is included. Future editions are not included unless the product listing says otherwise.

YOUR LEARNING ROUTE

Contents

01 / Define the metric before the query

02 / Create a complete practice dataset

03 / Filter dates and handle missing values

04 / Aggregate at a defensible grain

05 / Join without multiplying revenue

06 / Build a daily report and running total

07 / Rank customers and define ties

08 / Calculate repeat purchase responsibly

09 / Quality checks that catch real defects

10 / Read a plan before adding indexes

11 / Capstone: a reconciled monthly brief

Reference desk / Official documentation and next steps

FIELD GUIDE / 01

Define the metric before the query

A request for "monthly sales" contains several unresolved choices. Does sales mean submitted orders, payments captured, invoices issued or revenue recognised? Which date determines the month? Are cancellations, tax and shipping included? Write those decisions before touching the tables.

Our practice contract is deliberately narrow: count orders whose status is paid; use order_date as the reporting date; calculate quantity multiplied by unit_price less the line-level discount. A discount of 10 is a currency amount for the whole line, not a percentage. Order totals and line totals must not be mixed.

The grain rule

One row of orders represents one order. One row of lines represents one order line. A customer can have several orders, and an order can have several lines. Aggregating after a join is safe only when the joined rows still represent the unit being measured.

Record the contract

Metric: paid merchandise revenue.
Population: orders with status = 'paid'.
Formula: SUM(qty * unit_price - discount).
Time basis: order_date, represented as a date.
Exclusions: pending/cancelled orders, tax, shipping and unmodelled returns.

Practice and answer

A colleague wants order count from the joined line table. COUNT(*) counts lines. COUNT(DISTINCT order_id) counts orders; better still, count orders before joining unless a line condition is genuinely required. Store this distinction beside the dashboard measure.

FIELD GUIDE / 02

Create a complete practice dataset

Run this setup in one SQL session. TEMP tables are session-scoped and keep the exercise separate from persistent application tables. The keys prevent duplicate customer IDs and duplicate line numbers within an order. NUMERIC keeps currency arithmetic exact for these examples.

```
CREATE TEMP TABLE customers (
 id int PRIMARY KEY, name text, region text);
INSERT INTO customers VALUES
 (1,'Asha','West'),(2,'Bilal','South'),(3,'Chen','West');
CREATE TEMP TABLE orders (
 id int PRIMARY KEY, customer_id int REFERENCES customers,
 order_date date, status text);
INSERT INTO orders VALUES
 (101,1,'2026-01-03','paid'),
 (102,1,'2026-01-10','paid'),
 (103,2,'2026-01-10','paid'),
 (104,3,'2026-01-11','cancelled'),
 (105,2,'2026-02-02','paid');
CREATE TEMP TABLE lines (
 order_id int REFERENCES orders, line_no int,
 qty int CHECK(qty > 0), unit_price numeric(12,2),
 discount numeric(12,2) DEFAULT 0,
 PRIMARY KEY(order_id,line_no));
INSERT INTO lines VALUES
 (101,1,2,100,10),(101,2,1,50,0),
 (102,1,1,120,0),(103,1,3,40,0),
 (104,1,1,500,0),(105,1,2,75,0);
```

Establish a baseline

There are three customers, five orders and six lines. Four orders are paid. Their line amounts are 190, 50, 120, 120 and 150; paid revenue is 630. The cancelled order contributes 500 to all-order line amount but zero to the metric. If your result differs, stop and inspect the setup before adding complexity.

FIELD GUIDE / 03

Filter dates and handle missing values

Use half-open time intervals: include the first instant of the period and exclude the first instant of the next period. This pattern works for dates and translates well to timestamps. Avoid comparing a timestamp to an end-of-day string with guessed fractional precision.

```
SELECT id, customer_id, order_date
FROM orders
WHERE status = 'paid'
  AND order_date >= DATE '2026-01-01'
  AND order_date < DATE '2026-02-01'
ORDER BY order_date, id;
```

The result contains orders 101, 102 and 103. ORDER BY includes id so same-day rows are deterministic. For timestamp data, explicitly agree on a reporting time zone before deriving dates; otherwise a payment near midnight can move between months in different reports.

NULL is a state, not a zero

NULL means unknown or absent. WHERE discount = NULL will not select missing discounts; use IS NULL. SUM ignores NULL inputs, while COUNT(*) counts rows regardless. A missing price should fail a quality check rather than become free merchandise through an indiscriminate COALESCE.

Practice and answer

If a nullable discount means "no discount recorded, equivalent to zero" in an agreed contract, qty * unit_price - COALESCE(discount, 0) is reasonable. If NULL means an incomplete transaction awaiting review, coercing it to zero overstates confidence. The correct SQL depends on that business meaning. Add NOT NULL to the practice schema when the production contract requires complete values.

FIELD GUIDE / 04

Aggregate at a defensible grain

Start by forming one row per paid order. This intermediate result becomes a reusable contract: the next report can join customer attributes without counting the same order twice. It also makes reconciliation much easier than a single deeply nested expression.

```
WITH paid_orders AS (
 SELECT o.id, o.customer_id, o.order_date,
        SUM(l.qty*l.unit_price-l.discount) AS revenue
 FROM orders o
 JOIN lines l ON l.order_id=o.id
 WHERE o.status='paid'
 GROUP BY o.id, o.customer_id, o.order_date
)
SELECT COUNT(*) AS paid_order_count,
       SUM(revenue) AS revenue,
       ROUND(AVG(revenue),2) AS average_order_value
FROM paid_orders;
```

Expected output is four orders, revenue 630 and average order value 157.50. AVG on individual line amounts would produce 126 instead; it answers a different question because there are five paid lines.

HAVING versus WHERE

WHERE removes rows before aggregation. HAVING removes groups after aggregation. To find customers whose paid revenue exceeds 300, filter paid rows in WHERE, group by customer_id and apply HAVING SUM(...) > 300. Asha qualifies with 360; Bilal has 270.

Review question

Why not SUM(DISTINCT line_amount) to cure duplicated joins? Two legitimate lines may have the same amount. DISTINCT would erase a real sale. Fix the relationship or aggregate the many-side before joining; do not use value deduplication as a substitute for keys.

FIELD GUIDE / 05

Join without multiplying revenue

A join creates one result for each matching pair. If one order has two lines and three support tickets, joining all three tables produces six rows for that order. Summing line revenue across those rows triples it. This is a relationship problem, not an arithmetic problem.

```
WITH customer_spend AS (
 SELECT o.customer_id,
        SUM(l.qty*l.unit_price-l.discount) AS revenue
 FROM orders o JOIN lines l ON l.order_id=o.id
 WHERE o.status='paid'
 GROUP BY o.customer_id
)
SELECT c.id, c.name, COALESCE(s.revenue,0) AS revenue
FROM customers c
LEFT JOIN customer_spend s ON s.customer_id=c.id
ORDER BY c.id;
```

Asha returns 360, Bilal 270 and Chen 0. LEFT JOIN preserves customers without paid orders. Putting a condition on s in the outer WHERE can accidentally remove those unmatched customers; decide whether the filter belongs inside the aggregate or in the join condition.

Three join checks

Measure row counts before and after joining. Check that the expected unique key remains unique. Reconcile the sum of the metric before and after adding dimension attributes. All three checks are needed: a row count can stay unchanged while amounts attach to the wrong dimension key.

Practice and answer

Add support-ticket count by customer. Aggregate tickets to one row per customer first, then join the two customer-level summaries. Do not join raw tickets directly to raw order lines.

FIELD GUIDE / 06

Build a daily report and running total

A running total is easier to reason about when the input already has one row per reporting day. Aggregate first; apply a window second. Specify a frame so future readers do not need to infer how ties are treated.

```
WITH daily AS (
 SELECT o.order_date AS day,
        SUM(l.qty*l.unit_price-l.discount) AS revenue
 FROM orders o JOIN lines l ON l.order_id=o.id
 WHERE o.status='paid'
 GROUP BY o.order_date
)
SELECT day, revenue,
 SUM(revenue) OVER (
  ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING
  AND CURRENT ROW) AS running_revenue
FROM daily ORDER BY day;
```

The daily values are 240 on January 3, 240 on January 10 and 150 on February 2. Running revenue is 240, 480 and 630. There is no row for a day with no paid order.

Missing days are not missing sales

A chart requiring every calendar day needs a calendar spine. Generate or store the dates, LEFT JOIN daily revenue and convert an unmatched aggregate to zero. Only do that after deciding that absence truly means zero rather than a failed ingestion batch.

Practice and answer

January's reported daily rows sum to 480. A daily average over these two active dates is 240; an average over all 31 January dates is about 15.48. Neither is automatically wrong. Label the denominator, because "average daily revenue" without it can mislead a manager.

FIELD GUIDE / 07

Rank customers and define ties

Ranking is a product decision. ROW_NUMBER gives each row a different ordinal. RANK shares positions for ties and leaves gaps. DENSE_RANK shares positions without leaving gaps. Pick the one that matches the question instead of selecting the most familiar function.

```
WITH spend AS (
 SELECT o.customer_id,
        SUM(l.qty*l.unit_price-l.discount) AS revenue
 FROM orders o JOIN lines l ON l.order_id=o.id
 WHERE o.status='paid'
 GROUP BY o.customer_id
)
SELECT customer_id,revenue,
 DENSE_RANK() OVER (ORDER BY revenue DESC) AS spend_rank,
 ROUND(100.0*revenue/SUM(revenue) OVER (),2) AS share_pct
FROM spend ORDER BY revenue DESC,customer_id;
```

Asha ranks first with about 57.14% of revenue. Bilal ranks second with about 42.86%. Customer id makes display ordering stable without changing the revenue-based rank.

Partition deliberately

Adding PARTITION BY region ranks within each region. It also changes the denominator if you partition the SUM window. A "regional share" and a "company share" should not share the same label. A customer belongs to a region at a point in time; if historical region matters, use a dated dimension rather than today's customer address.

Practice and answer

Suppose two customers each contribute 270. DENSE_RANK assigns both rank 2. ROW_NUMBER can assign 2 and 3, but requires a documented tie-breaker if the top-N output must be reproducible.

FIELD GUIDE / 08

Calculate repeat purchase responsibly

A repeat customer has at least two qualifying orders in the observed window. That is not the same as retention, which usually follows a cohort through later periods. A narrow date window can hide earlier purchases and mislabel an established customer as new.

```
WITH paid AS (
 SELECT customer_id, COUNT(*) AS order_count
 FROM orders WHERE status='paid'
 GROUP BY customer_id
)
SELECT COUNT(*) AS buying_customers,
 COUNT(*) FILTER (WHERE order_count>=2) AS repeat_customers,
 ROUND(100.0*COUNT(*) FILTER (WHERE order_count>=2)
       /NULLIF(COUNT(*),0),2) AS repeat_pct
FROM paid;
```

Both observed buying customers have two paid orders, so repeat_pct is 100. Chen is excluded from the denominator because Chen has no paid orders. Including all three registered customers would produce a different metric, 66.67%.

Sequence purchases

Use LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date,id) to retrieve the previous paid order. The gap is current date minus previous date. Exclude a customer's first order from the average gap rather than treating its missing previous date as zero days.

Practice and answer

For Asha the gap is seven days; for Bilal it is 23 days. Their mean observed repeat gap is 15 days. This small invented sample is useful for checking syntax, not for estimating a real customer's future behaviour.

FIELD GUIDE / 09

Quality checks that catch real defects

A query can run successfully and still be wrong. Save quality checks as executable queries with expected outcomes. Differentiate failures that invalidate the report from warnings that need investigation. Avoid hiding problems inside a dashboard tooltip.

```
-- Expected zero rows: invalid monetary lines.
SELECT * FROM lines
WHERE qty IS NULL OR qty<=0 OR unit_price IS NULL OR unit_price<0
   OR discount IS NULL OR discount<0
   OR discount>qty*unit_price;

-- Expected zero rows: orders without lines.
SELECT o.id FROM orders o
LEFT JOIN lines l ON l.order_id=o.id
GROUP BY o.id HAVING COUNT(l.order_id)=0;

-- Expected zero rows: duplicate order/line keys.
SELECT order_id,line_no,COUNT(*)
FROM lines GROUP BY order_id,line_no HAVING COUNT(*)>1;
```

The primary key already prevents duplicates in our practice table. Keeping the duplicate query is still useful when validating an incoming file before loading it. Referential constraints should back up checks where feasible; a report check alone does not protect every writer.

Reconcile in layers

Compare source row count to staged row count, staged paid revenue to order summary revenue, and order summary revenue to the final report. Record the same cutoff time for each comparison. A late payment arriving between snapshots otherwise looks like a defect.

Practice and answer

If every aggregate increases after a new dimension is joined, inspect duplicate dimension keys first. If only one day drops, inspect date filtering and ingestion completeness. Investigate the smallest failing boundary rather than rewriting the whole query.

FIELD GUIDE / 10

Read a plan before adding indexes

Performance changes need evidence. Use EXPLAIN to inspect the planned operations; EXPLAIN ANALYZE actually executes the statement. On writes or expensive queries, that distinction matters. Benchmark with representative data volume and a consistent cache state when comparing alternatives.

```
EXPLAIN
SELECT id,customer_id,order_date
FROM orders
WHERE status='paid'
  AND order_date>=DATE '2026-01-01'
  AND order_date<DATE '2026-02-01';
```

Our tiny dataset is too small to justify an index recommendation. A sequential scan may be perfectly appropriate. On a larger persistent table, a candidate index on status and order_date can help some access patterns, but selectivity, write overhead and other queries determine its value. Confirm with the actual plan and workload.

Prefer clear predicates

A range on order_date is generally easier to index than wrapping every row's date in a formatting function. Select only needed columns for large exports. Aggregate before a many-to-many join. Materialising a repeated expensive summary may help, but introduces refresh and freshness responsibilities.

Review checklist

Capture the original query, representative parameters, plan, execution time and row count. Change one thing. Verify the result set still matches, including NULL behaviour and ties. Then measure again. A faster query that changes the metric is a regression, not an optimisation.

FIELD GUIDE / 11

Capstone: a reconciled monthly brief

Deliver a report with one row per month, paid-order count, revenue and average order value. Keep the order-level aggregation visible so an analyst can trace each total back to its source rows.

```
WITH paid AS (
 SELECT o.id,o.order_date,
        SUM(l.qty*l.unit_price-l.discount) AS revenue
 FROM orders o JOIN lines l ON l.order_id=o.id
 WHERE o.status='paid'
 GROUP BY o.id,o.order_date
)
SELECT DATE_TRUNC('month',order_date)::date AS month,
 COUNT(*) AS orders,SUM(revenue) AS revenue,
 ROUND(AVG(revenue),2) AS aov
FROM paid GROUP BY 1 ORDER BY 1;
```

Answer key

January: 3 orders, 480 revenue, 160 AOV.
February: 1 order, 150 revenue, 150 AOV.
Across both months: 4 orders, 630 revenue, 157.50 AOV.

Do not average the monthly AOV values: (160 + 150) / 2 = 155 weights months equally, not orders. Recompute total revenue divided by total order count for the combined period.

Handoff package

Save the metric contract, setup SQL, report SQL and zero-row quality checks. Include the excluded cancelled order and the treatment of discounts in the note. Explain that returns and tax are outside this practice schema. A useful handoff lets another analyst reproduce the totals without asking what each expression was intended to mean.

KEEP LEARNING

Reference desk

Official references checked while preparing this edition. Use these pages for syntax, feature availability and the latest product details. The exercises and business scenarios in this book are original.

PostgreSQL tutorial: queries and joins

https://www.postgresql.org/docs/current/tutorial-join.html

PostgreSQL tutorial: aggregate functions

https://www.postgresql.org/docs/current/tutorial-agg.html

PostgreSQL tutorial: window functions

https://www.postgresql.org/docs/current/tutorial-window.html

PostgreSQL: using EXPLAIN

https://www.postgresql.org/docs/current/using-explain.html

Make the next step observable

Recreate the final brief with a new month and one additional customer. Predict each changed total before running the query. Keep your reconciliation checks next to the SQL so correctness remains visible as the report grows.

Questions about your purchased edition? Visit dattasable.com/contact and include your order number. Independently published; no endorsement by the software vendors named in this guide is implied.